

INDIA_CRYPTO_SOC

Architecture & Microarchitecture Presenter Script

AES-CA Accelerated RISC-V SoC for Indian Government Document Encryption

Aadhaar PDF · Government Document · Medical Record

Module	Description	Address
RV32IM Core	5-Stage Pipeline + Attack Protections	—
AES ISA Ext	Custom-0 Sideband, 8 AES Instructions	—
AES-CA Accel	AES-256 with 4 Cellular Automaton Rounds	0x3000_0000
MS-TRNG	Metastable FF TRNG, SP 800-90B Health	0x3000_1000
India PDF Engine	17-State FSM, AES-CTR + CA-HMAC	0x3000_2000
Secure UART	PULP apb_uart + 5 Security Features	0x4000_0000
Secure I2C	PULP apb_i2c + Whitelist + Recovery	0x4000_1000
Secure SPI	PULP apb_spi_master + CS Guard	0x4000_2000
AXI Firewall	Per-(master,slave) Access Control	Per slave

Estimated delivery time: 12–15 minutes · 11 slides

Table of Contents

1	Title Slide INDIA_CRYPTOSOC overview & target applications
2	SoC Block Diagram Top-level architecture: CPU, crossbar, memory, crypto, peripherals
3	Memory Map & Address Space 8-slave address table, AXI masters, firewall defaults
4	RV32IM Core Microarchitecture 5-stage pipeline, 4 attack protections, custom CSRs
5	AES-CA Accelerator 4-CA round operations, CTR mode, direct interface
6	Encrypt & Decrypt Flow Step-by-step software sequences for both directions
7	MS-TRNG + India PDF Engine Entropy pipeline, packet format, APP_MODE register
8	AXI Fabric & Firewall Crossbar topology, permission table, firewall registers
9	Secure Peripherals UART, I2C, SPI — PULP IP + security wrappers
10	Attack Protection Summary Consolidated security feature grid
11	Target Applications Aadhaar, GovDoc, Medical — content & auth levels

How to Use This Script

This document is the complete presenter script for the INDIA_CRYPTOSOC Architecture and Microarchitecture presentation. Each section corresponds to one slide in the deck (*INDIA_CRYPTOSOC_Architecture.pptx*).

Each section contains three parts:

- **What's on this slide** — a quick list of the visual elements and data shown on that slide, so you know exactly what the audience is looking at.
- **Technical Reference** — key register addresses, signal names, or configuration values shown as a code block. Use these to answer follow-up questions without searching.
- **Presenter Script** — the word-for-word (or near-word-for-word) narration. Read naturally; do not read it verbatim if the audience is technical — adapt to the room.

Timing guide: Slides 1–3 are overview slides (~1 min each). Slides 4–8 are technical deep-dives (~2 min each). Slides 9–11 are summary/application slides (~1 min each). Total: 12–15 minutes with 2–3 minutes reserved for questions.

Title Slide — INDIA_CRYPTO_SOC

WHAT'S ON THIS SLIDE

- Project name: INDIA_CRYPTO_SOC in large white text
- Subtitle: AES-CA Accelerated RISC-V SoC for Indian Government Document Encryption
- Three application cards: Aadhaar PDF, Gov Document, Medical Record
- Module summary tag: RV32IM Core · AES-256 4-CA · MS-TRNG · AXI Firewall · PULP Peripherals
- Dark navy background with saffron and green accents (India flag palette)

PRESENTER SCRIPT

Good morning, everyone. Today I'll walk you through INDIA_CRYPTO_SOC — a custom hardware SoC built from the ground up specifically for encrypting and decrypting Indian government documents.

The three core use cases are Aadhaar PDFs, which contain biometric and identity data managed by UIDAI; government documents on DigiLocker and the NIC ecosystem under the IT Act 2000; and medical records under ABDM and the HL7 FHIR standard.

What makes this SoC different from a general-purpose encryption engine is that every layer — from the CPU to the bus fabric to the peripherals — has been hardened specifically for this threat model. We're not just adding AES on top of an existing SoC. We've designed the security in from the instruction set down to the bus arbitration logic.

The key modules are a hardened RV32IM processor, a novel AES-256 accelerator using cellular automata, a metastable true random number generator, an AXI-Lite hardware firewall, and three attack-protected peripherals sourced from the open-source PULP platform.

SoC Architecture — Block Diagram

WHAT'S ON THIS SLIDE

- CPU Subsystem box: RV32IM Core (left) + AES ISA Ext (right)
- AXI-Lite Crossbar spanning full width below CPU (2M × 8S)
- Three slave columns: Memory (Boot ROM, SRAM), Crypto (AES-CA, TRNG, PDF Engine), Peripherals (UART, I2C, SPI)
- Crypto and Peripheral columns labelled [Firewall Protected]
- Connector lines from CPU to crossbar, crossbar to each column

```
Masters:  m0 = CPU dmem (high priority, trusted)
          m1 = PDF DMA (low priority, semi-trusted)
Slaves:   s0 Boot ROM  s1 SRAM  s2 AES-CA  s3 TRNG  s4 PDF
          s5 UART      s6 I2C   s7 SPI
CPU imem -> SRAM Port A (direct, bypasses crossbar)
```

PRESENTER SCRIPT

Here is the top-level architecture. Starting at the top — the CPU subsystem contains the RV32IM core on the left and the AES ISA extension on the right. These two blocks are tightly coupled: the RV32IM core issues a custom-0 opcode and immediately receives the AES result back through a combinational sideband within the same clock cycle. No memory round-trip, no AXI transaction overhead.

Everything communicates through the AXI-Lite crossbar, which sits as a horizontal fabric between the CPU and all the slaves. The crossbar has two masters: master zero is the CPU data bus with high priority, and master one is the DMA channel inside the PDF engine with lower priority. The CPU instruction bus is wired directly to SRAM Port A, completely bypassing the crossbar — this keeps the fetch path latency minimal and guarantees the CPU always has instruction bandwidth regardless of DMA activity.

Below the crossbar are three groups. On the left, memory: the Boot ROM where the CPU starts executing, and the 128-kilobyte SRAM which serves both the CPU data path and the DMA master. In the middle, the crypto engines — the AES-CA accelerator, the TRNG, and the India PDF engine. On the right, the three secure peripherals. Critically, everything from the crypto engines rightward sits behind individual hardware firewalls. I'll detail the firewall on slide eight.

SLIDE 3

Memory Map & Address Space

WHAT'S ON THIS SLIDE

- Full 8-row address table: range, size, module name, slave ID, firewall flag
- AXI Masters info box: CPU m0 (trusted, HIGH) and DMA m1 (semi-trusted, LOW)
- Firewall Default Permissions table: CPU=YES, DMA=NO for s2–s7
- Footer note: DMA can access SRAM (s1) directly without a firewall

0x0000_0000	-	0x0000_7FFF	32 KB	Boot ROM	s0	no firewall
0x2000_0000	-	0x2001_FFFF	128 KB	SRAM	s1	no firewall
0x3000_0000	-	0x3000_0FFF	4 KB	AES-CA Accel	s2	FIREWALL
0x3000_1000	-	0x3000_1FFF	4 KB	MS-TRNG	s3	FIREWALL
0x3000_2000	-	0x3000_2FFF	4 KB	India PDF Engine	s4	FIREWALL
0x4000_0000	-	0x4000_0FFF	4 KB	Secure UART	s5	FIREWALL
0x4000_1000	-	0x4000_1FFF	4 KB	Secure I2C	s6	FIREWALL
0x4000_2000	-	0x4000_2FFF	4 KB	Secure SPI	s7	FIREWALL

PRESENTER SCRIPT

The address space is divided into eight slaves. The Boot ROM sits at address zero and is 32 kilobytes — the CPU always resets here and begins executing the secure boot sequence. The SRAM is 128 kilobytes starting at 0x2000_0000, and it's the main working memory for both the CPU and the DMA engine.

From 0x3000_0000 onwards are the crypto engines, each with a 4-kilobyte page. The AES-CA accelerator is at 0x3000_0000, the TRNG at 0x3000_1000, and the PDF engine at 0x3000_2000. The peripherals occupy the 0x4000 range in the same pattern — UART, I2C, then SPI.

Everything from slave two upwards is firewall-protected. By default, after reset, only the CPU can reach any of those slaves. The DMA master is denied access to the crypto and peripheral regions. The DMA can however access SRAM directly — that's by design, because the PDF engine needs to stream large amounts of data in and out of SRAM without creating bus congestion for the CPU. If the firmware needs to grant the DMA access to any other slave — for example during a provisioning phase — the CPU writes the firewall permission table at runtime.

SLIDE 4

RV32IM Core — Microarchitecture

WHAT'S ON THIS SLIDE

- 5-stage pipeline diagram: IF → ID → EX → MEM → WB with stage sub-labels
- 2×2 grid of attack protection cards: Instruction Parity, PC Range Guard, Register ECC, Watchdog
- Right panel: Custom CSRs (0xBC0–0xBC2), M-Extension instruction list, AES Custom-0 Sideband signals

Custom CSRs:

```
0xBC0  pc_min_guard  - minimum valid PC (default 0x0000_0000)
0xBC1  pc_max_guard  - maximum valid PC (default 0x2001_FFFF)
0xBC2  wdt_reload   - 24-bit watchdog reload value
```

AES Sideband signals:

```
custom_valid, custom_instr[31:0], custom_rs1[31:0], custom_rs2[31:0]
custom_rd[31:0] (result), custom_rd_valid (asserted same cycle)
```

AES ISA Instructions:

```
aes.esb  aes.emx  aes.esr  (forward: SubBytes, MixColumns, ShiftRows)
aes.dsb  aes.dmx  aes.disr (inverse: SubBytes, MixColumns, ShiftRows)
aes.xk   aes.rcon                (KeyExpand, RoundConstant)
```

PRESENTER SCRIPT

The CPU is a classic five-stage pipeline — Instruction Fetch, Decode, Execute, Memory access, and Write-back. On top of this standard RISC-V RV32IM pipeline we've added four independent hardware attack protections.

First, instruction parity. Every instruction word in memory carries a parity bit. When the fetch stage pulls an instruction, it recomputes the parity. A mismatch — which would indicate either a fault injection or a memory corruption attack — immediately raises an illegal-instruction trap. The CPU never executes a potentially corrupted instruction.

Second, the PC range guard. Two custom CSRs — 0xBC0 and 0xBC1 — define the minimum and maximum valid program counter values. If the CPU ever fetches from outside that window, execution halts. This prevents return-oriented programming attacks and hardware-assisted fault injection that tries to redirect the PC into data memory.

Third, register file ECC. Every nibble — four bits — in all 32 general-purpose registers is protected with a Hamming(7,4) code. Single-bit errors are silently corrected in hardware. Double-bit errors are flagged as fatal, triggering a CPU reset. This addresses row-hammer style attacks on the register array.

Fourth, the 24-bit watchdog timer. It reloads on every committed instruction. If the CPU stalls — either due to a hung peripheral, a deadlock condition, or an attacker trying to hold a cryptographic operation in an intermediate state — the watchdog resets the system. There is no way to leave a half-completed AES operation exposed by stalling the pipeline.

On the right, you can see the AES custom sideband. When the RISC-V core encounters a custom-0 opcode, it broadcasts the instruction word and both source register values to the AES ISA extension. The extension is purely combinational — it computes the result and puts it back on the custom_rd bus in the same cycle. The CPU treats this exactly like a single-cycle ALU operation. This gives software a way to implement key expansion and individual AES round operations without touching the AXI bus at all.

SLIDE 5

AES-CA Accelerator — 4-CA Microarchitecture

WHAT'S ON THIS SLIDE

- 4 CA stage blocks stacked vertically with numbered badges (1–4)
- CA-1: SubBytes + Neighbour Perturbation (red)
- CA-2: ShiftRows (orange)
- CA-3: MixColumns over GF(2⁸) (blue)
- CA-4: AddRoundKey + Rule-90 CA (green)
- Right panel: CTR Mode formulas + Direct Interface signal list
- Bottom bar: AES-256 14 rounds · CTR Mode · same HW for enc/dec

```
CA-1: 128-bit state -> S-box lookup -> XOR with row neighbours (CA rule)
CA-2: row i shifted left by i bytes (0, 1, 2, 3)
CA-3: MDS matrix multiply in GF(2^8), polynomial x^2+x+1
CA-4: XOR with 256-bit round key; Rule-90: state[i]=state[i-1]^state[i+1]
Rounds: 14 total for AES-256
Direct interface ports:
  direct_key[255:0]   direct_din[127:0]   direct_start
  direct_decrypt      direct_dout[127:0]  direct_done
```

PRESENTER SCRIPT

The AES accelerator is where the cryptographic heavy lifting happens. Instead of implementing the four standard AES transformations as static combinational logic, we map each one to a cellular automaton operation. Each CA stage takes the 128-bit cipher state as its cell array and applies a local neighbourhood rule simultaneously across all cells in parallel.

CA-1 performs SubBytes with a twist. Each byte is substituted through the standard AES S-box, and then XOR'd with its horizontal row neighbours according to a CA-1 rule. This adds a data-dependent perturbation that makes power analysis attacks significantly harder — the intermediate state at every byte position depends on its neighbours, not just its own S-box output.

CA-2 is ShiftRows — the standard AES byte rotation. Row zero is unchanged, row one shifts left by one byte, row two by two, row three by three.

CA-3 is MixColumns over GF(2 to the power of 8). This is the full MDS matrix multiplication that provides diffusion across each 4-byte column. The irreducible polynomial is x-squared plus x plus 1.

CA-4 applies AddRoundKey and then runs a Rule-90 cellular automaton across the 128-bit state. Rule-90 means each cell becomes the XOR of its two immediate neighbours. This adds an extra layer of diffusion after the key mixing step, increasing the avalanche effect.

The critical point for software is CTR mode. In counter mode, the cipher always runs in the forward direction — we encrypt a counter block, then XOR the output with either plaintext or ciphertext. The XOR operation is symmetric, so the exact same hardware performs both encryption and decryption. No inverse S-box, no inverse MixColumns. This halves the hardware area compared to supporting both directions natively.

The PDF engine talks to the accelerator through six direct wires — no AXI overhead, no bus arbitration. This gives the PDF engine single-clock-cycle access to start a new AES block operation the moment the previous one completes.

SLIDE 6

Encryption & Decryption Flow

WHAT'S ON THIS SLIDE

- Two-column layout: ENCRYPT (left, green header) and DECRYPT (right, red header)
- Six numbered encrypt steps, five decrypt steps + auth result split box
- auth_ok (green) and auth_fail (red) outcome boxes at bottom right

ENCRYPT sequence:

```
CTRL = 0x05 (START | IRQ_EN)
Poll STATUS [0]=done, [2]=auth_ok
Read RESULT_TAG[0..3] @ 0x3000_2068
```

DECRYPT sequence:

```
Write EXP_TAG[0..3] @ 0x3000_2048
CTRL = 0x07 (START | DECRYPT | IRQ_EN)
auth_ok=1 -> plaintext valid at DST_ADDR
auth_fail=1 -> hardware has zeroed DST_ADDR region
```

PRESENTER SCRIPT

This slide shows the complete software sequence for both directions. Let's walk through encryption first.

Step one — collect entropy. The CPU polls the TRNG STATUS register until the valid bit is set, then reads the DATA register twelve times. Eight of those 32-bit words fill the 256-bit AES key registers. The remaining four fill the 128-bit nonce. This is the only source of key material — no software-generated pseudo-random keys are accepted.

Steps two and three configure the PDF engine. The CPU writes the application mode, the source and destination addresses in SRAM, the document length, document metadata including DOC_ID, timestamp, and auth level, and then loads the key and nonce into the engine's registers.

Step four — writing 0x05 to CTRL — starts the engine and enables the interrupt. The CPU then sits in a wait-for-interrupt state. When the engine finishes, it asserts the IRQ line connected to the CPU's external interrupt input.

Step five is what the engine does autonomously. It streams an encrypted header, then the full PDF binary — every byte, including embedded images, fonts, and metadata — through the AES-CTR cipher, and appends the CA-HMAC authentication tag at the end of the ciphertext.

Step six — the CPU wakes from the interrupt, reads the computed RESULT_TAG registers, and stores the 128-bit tag alongside the ciphertext. This tag is the proof of authenticity for the document.

Decryption is the mirror. The key and nonce are reloaded — either from secure storage or a key exchange protocol. The expected tag from storage goes into EXP_TAG. Writing 0x07 starts decryption. When the interrupt fires, the CPU checks STATUS. If auth_ok is set, the plaintext at DST_ADDR is valid and the document is authentic. If auth_fail is set instead, the hardware has already zeroed the entire destination region — the CPU cannot access partially decrypted or tampered data, even by accident.

MS-TRNG & India PDF Engine

WHAT'S ON THIS SLIDE

- Left half: MS-TRNG — 4-stage pipeline (Metastable FF → Von Neumann → LFSR → Health Tests)
- TRNG AXI-Lite register map @ 0x3000_1000
- Right half: India PDF Engine — 17-State FSM
- Output packet format: Encrypted Header | Encrypted PDF Binary | CA-HMAC Tag
- PDF Engine register map @ 0x3000_2000
- Three APP_MODE cards: Aadhaar, GovDoc, Medical

```

TRNG Registers @ 0x3000_1000:
0x00 STATUS [0]=valid
0x04 DATA 32-bit random word (read clears valid bit)
0x08 HEALTH [1]=APT_fail [0]=RCT_fail (write to clear)
0x0C CTRL [0]=enable [1]=rst_health

PDF Engine Key Registers @ 0x3000_2000:
0x000 CTRL 0x004 APP_MODE 0x008 STATUS
0x00C SRC_ADDR 0x010 DST_ADDR 0x014 PDF_LEN
0x018 NONCE[0..3] (128-bit, 4 words)
0x028 KEY[0..7] (256-bit, 8 words)
0x048 EXP_TAG[0..3] (decrypt: expected HMAC)
0x068 RESULT_TAG[0..3] (encrypt: computed HMAC)

```

PRESENTER SCRIPT

On the left — the metastable TRNG. The design uses eight independent channels, each consisting of a pair of flip-flops clocked on opposite edges. When a flip-flop is deliberately set up with insufficient hold time relative to the incoming data, it enters a metastable state. The time it takes to resolve — either to a zero or a one — is governed by thermal noise at the transistor level. That resolution time is the source of entropy.

The eight raw bitstreams are fed into a Von Neumann debiaser, which pairs consecutive bits and outputs a one only when they differ — this removes any systematic bias toward zero or one. The debiased stream then passes through a Galois LFSR for whitening, spreading the entropy uniformly across all bit positions. Finally, two continuous health tests from NIST SP 800-90B run in hardware — the Repetition Count Test catches a stuck channel that produces the same value repeatedly, and the Adaptive Proportion Test detects a channel with severely skewed probability.

On the right — the India PDF Engine. Every output packet has exactly three sections. The encrypted header carries the application mode, document ID, timestamp, and auth level — metadata that lets the recipient verify the document type without decrypting the body. The encrypted binary stream is the raw PDF bytes — text, images, fonts, form data, everything — streamed through the AES-CTR cipher in 16-byte blocks. Finally, the CA-HMAC tag is a 128-bit integrity code computed by running a Rule-30 cellular automaton over the cipher state for eight iterations. This tag is appended at the end of every encrypted document.

The APP_MODE register is the only structural difference between the three document types. Mode zero for Aadhaar, mode one for government documents, mode two for medical records. The cipher operation is identical in all three cases — what changes is the metadata written into the encrypted header and the auth level stored for access control decisions.

SLIDE 8

AXI-Lite Fabric & Firewall

WHAT'S ON THIS SLIDE

- Left: Crossbar topology — m0/m1 masters, AXI crossbar block, s0–s7 slave columns
- SRAM (s1) note: DMA accesses directly, no firewall
- Right: Firewall block properties + full register map

```
Firewall Registers (one instance per slave s2–s7):
0x00  PERM_TABLE      [31:0] bit[slave*4+master] = allow
0x04  LOCK_TABLE      write 0xDEADBEEF to make read-only
0x08  LAST_DENIED_ADDR [31:0] address of last denied transaction
0x0C  LAST_DENIED_MASTER [1:0] master_id of denied transaction
0x10  DENY_COUNT      [15:0] cumulative deny count (write to clear)
Default PERM_TABLE after reset:
CPU (master 0) on all slaves s2..s7 : ALLOWED
DMA (master 1) on all slaves s2..s7 : DENIED
```

PRESENTER SCRIPT

The AXI-Lite crossbar is a standard 2-master, 8-slave fabric. The CPU data bus is master zero and has priority — when both the CPU and the DMA want to access the same slave simultaneously, the CPU wins. The PDF engine's DMA is master one and accepts lower priority because it can tolerate one-cycle latency between block operations. The crossbar is implemented in RTL with a fixed arbitration policy — no dynamic priority inversion is possible.

Sitting between the crossbar and each protected slave is an independent firewall instance. The firewall holds a 32-bit permission table where each bit corresponds to one master-slave pair. Bit zero of that register means: master zero is allowed to access this slave. Bit one means: master one is allowed.

After secure boot completes, the boot firmware writes 0xDEADBEEF to the LOCK_TABLE register of every firewall. This makes the permission table permanently read-only for the remainder of the power cycle. An attacker cannot reconfigure the firewall from software, even with full CPU control, once the lock has been set.

Every denied transaction is logged. The LAST_DENIED_ADDR register captures the address that was blocked, LAST_DENIED_MASTER records which master attempted the access, and a 16-bit DENY_COUNT accumulates over the session. The CPU can read these registers to detect intrusion attempts — if the deny counter is non-zero when it wasn't expected to be,

something is probing the bus fabric. The denied master receives an AXI SLVERR response, which the CPU can trap as a machine-mode exception.

SLIDE 9

Secure Peripherals — UART, I2C, SPI

WHAT'S ON THIS SLIDE

- Three-column layout: Secure UART (teal), Secure I2C (cyan-teal), Secure SPI (blue)
- Each column: source IP label, five security feature boxes, register summary
- All three use PULP platform IP with axil_to_apb bridge

```
UART key registers @ 0x4000_0000:  
  0x00 DATA  0x04 STATUS[7:0]  0x08 BAUD  0x10 LOCK (write 0xA5)  
I2C key registers @ 0x4000_1000:  
  0x18 WL_CTRL [1:0]=index [8]=WL_EN  
  0x1C WL_ADDR [6:0] 7-bit address entry  
SPI key registers @ 0x4000_2000:  
  0x14 MAX_LEN [7:0] max transfer bytes (enforced in hardware)  
  0x18 CS_LOCK [3:0] write-once per-CS disable
```

PRESENTER SCRIPT

All three peripherals use production-proven IP from the PULP platform — open-source silicon that has been silicon-validated in multiple academic and industrial chips from ETH Zurich. We wrap each with an AXI-Lite to APB bridge and a security layer that adds attack protections specific to each interface.

The UART security layer adds four features on top of the standard PULP `apb_uart`. A three-sample glitch filter on the RX line votes across three consecutive samples and only accepts a bit transition if all three agree. This eliminates single-cycle glitch injection on the receive path. The baud rate can be locked by writing 0xA5 to the LOCK register — after that, no software write can change the divisor, preventing a man-in-the-middle attack that would resync the receiver to an attacker-controlled baud rate. Frame errors — which can indicate an attacker toggling the line at an unexpected rate — are counted and flagged in the STATUS register's ATTACK_ALERT bit.

The I2C security layer adds a four-entry address whitelist. The CPU programs up to four 7-bit device addresses into the whitelist registers, and enables the filter. After that, the master will only initiate transactions to those four addresses. Any other target address is blocked in hardware — even if software writes a different ADDR register value. This prevents an attacker who gains code execution from exfiltrating data to an unexpected I2C device. Additionally, the bus-stuck detector monitors the SCL line. After 500 thousand clock cycles without a completed transaction, it asserts BUS_STUCK in STATUS and automatically executes a nine-pulse SCL recovery sequence — the I2C specification's defined procedure for clearing a stuck slave.

The SPI security layer has three protections. The hardware chip-select guard means the SPI CS pins are controlled exclusively by the peripheral state machine, not by a software-writable output register. Firmware cannot hold CS asserted across a transaction boundary, which would allow timing-based side-channel attacks. The MAX_LEN register caps the number of bytes per transfer — any transaction that exceeds this limit is terminated mid-way by the hardware. And the CS_LOCK register allows individual chip-select pins to be permanently disabled — after provisioning, if a given SPI device is no longer needed, its CS pin can be locked off and cannot be re-enabled without a full power cycle.

SLIDE 10

Attack Protection Summary

WHAT'S ON THIS SLIDE

- 2x2 quadrant grid on dark navy background
- Top-left: CPU Core Protections (navy card)
- Top-right: AXI Firewall (dark green card)
- Bottom-left: Secure UART & I2C (navy card)
- Bottom-right: SPI & PDF Engine (dark card)

PRESENTER SCRIPT

This slide consolidates the complete security posture of the SoC into one view. Four quadrants, four attack surfaces.

The CPU core has four hardware protections: instruction parity trapping on fetch, PC range guarding via custom CSRs, register file ECC with Hamming codes, and a watchdog that resets on any CPU stall. Together these address fault injection, control-flow hijacking, and clock glitching attacks on the processor.

The AXI firewall adds a system-level access control layer. Every master-slave pair has an independent permission bit. The permission table is write-once after secure boot. Denied accesses are logged with full address and master ID. An unauthorized access returns SLVERR to the requesting master and increments the deny counter — giving the security firmware a way to detect probing.

For the peripherals — UART and I2C together address physical-layer attacks on the communication interfaces. Glitch filters on both RX lines, baud-rate locking on UART, address whitelisting and bus-stuck recovery on I2C. SPI adds the hardware CS guard and per-CS lockout to prevent chip-select manipulation.

Finally, the PDF engine provides document-level integrity for every operation. The CA-HMAC authentication tag means every encrypted document is self-authenticating. On decryption, if the tag doesn't match — whether because the ciphertext was tampered with in storage, modified in transit, or fed to the wrong key — the hardware zeroes the output before the CPU sees it. There is no software path that can extract a partially decrypted plaintext.

Target Applications

WHAT'S ON THIS SLIDE

- Three tall application cards: Aadhaar (navy), GovDoc (dark green), Medical (dark purple)
- Each card: large decorative number (01/02/03), APP_MODE badge, title, authority, content list
- Auth levels: Aadhaar=restricted, GovDoc=secret, Medical=top-secret

```
APP_MODE = 0  Aadhaar PDF    – Auth-level: restricted (1)
  Content: face JPEG, fingerprint, iris scan, UID, QR code
  Authority: UIDAI
APP_MODE = 1  Gov Document  – Auth-level: secret (2)
  Content: scanned pages, digital signatures, official seals
  Authority: NIC / DigiLocker / IT Act 2000
APP_MODE = 2  Medical Record – Auth-level: top-secret (3)
  Content: DICOM slices, X-rays, MRI/CT, lab reports, prescriptions
  Authority: ABDM / National Health Authority
```

PRESENTER SCRIPT

Finally — the three target applications. These are not three separate software stacks. They share identical hardware. The only structural difference between them is the APP_MODE register value and the auth level stored in the document header.

Aadhaar PDFs, issued by UIDAI, are the most widely distributed sensitive government documents in India. They contain a face photograph, fingerprint minutiae data, iris scan templates, the twelve-digit UID number, and a QR code encoding a compressed biometric profile. All of this is a single binary blob. The PDF engine treats it exactly that way — it doesn't parse the PDF, it doesn't extract the image, it encrypts every byte from offset zero to PDF_LEN as a contiguous stream. Auth-level restricted — only intended for government verifiers.

Government documents — those distributed through NIC portals and DigiLocker under the IT Act 2000 — contain scanned pages of official documents, digital signatures, stamp images, and embedded form data. These may be larger files than an Aadhaar card. The PDF_LEN register handles any size up to the SRAM capacity, and for larger documents firmware chunks the file. Auth-level secret.

Medical records under the Ayushman Bharat Digital Mission and the HL7 FHIR standard are the most security-sensitive. A DICOM-formatted imaging file containing X-ray, MRI, or CT scan data can be tens of megabytes. The PDF engine handles this via chunked DMA transfers — the engine reads blocks from SRAM, encrypts them, and writes them back, moving through the document progressively. The CA-HMAC tag covers the entire document, not individual chunks, so integrity is guaranteed end-to-end. Auth-level top-secret.

In all three cases, the guarantee is the same: every byte of every file — including every embedded image, every scanned page scan, every DICOM slice — is encrypted as a single cryptographic unit with a single authentication tag. There is no partially-encrypted PDF, no image that's been left in plaintext, and no situation where a valid decryption can occur without an intact, matching CA-HMAC tag. Thank you.

ESTIMATED DELIVERY: 12–15 MINUTES

Allow 2–3 minutes at the end for questions. Common questions: key exchange protocol (answer: pre-shared or wrapped keys loaded into KEY regs by secure boot firmware); TRNG entropy rate (answer: ~8 Mbit/s before health tests, ~4 Mbit/s after Von Neumann debiasing); SRAM size limit (answer: 128 KB per chunk, firmware handles larger files); Firewall granularity (answer: per master-per slave, not per address range — once you pass the firewall, the full 4 KB page is accessible).

File: INDIA_CRYPTOSOC_Architecture.pptx · Path: /Users/aasheiksaran/Desktop/Productivity/INDIA_CRYPTOSOC/